Workload Management

DIRAC and DiracX as compute backends

Alexandre F. Boyer · CERN

12th DIRAC(X) Users' Workshop · FZU Prague · 13–16 October 2026



ARM and GPUs are spreading

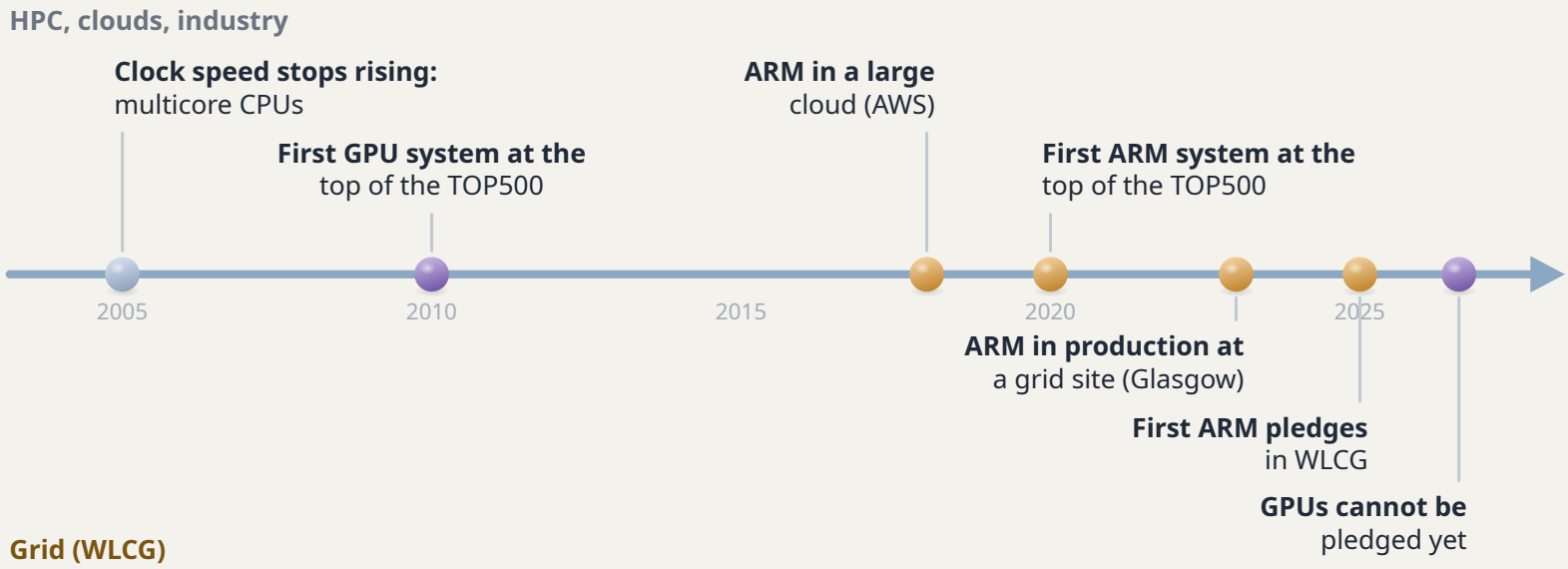


276 / 500
TOP500 systems with GPUs, June 2026 (110 in 2018)

35
ARM systems in the TOP500 (5 in 2020)

Our communities
run mostly on grid sites, and use a few HPCs opportunistically.

HPC and clouds see them first



The grid follows, years later: sites buy from the same market, once the experiments can use the hardware.

The end of two decades of homogeneity

> *Sites buy from the same market as everyone else: "Hardware technology evolution now is dominated by the needs of AI" (A. Sciabà, CERN IT, July 2026).*

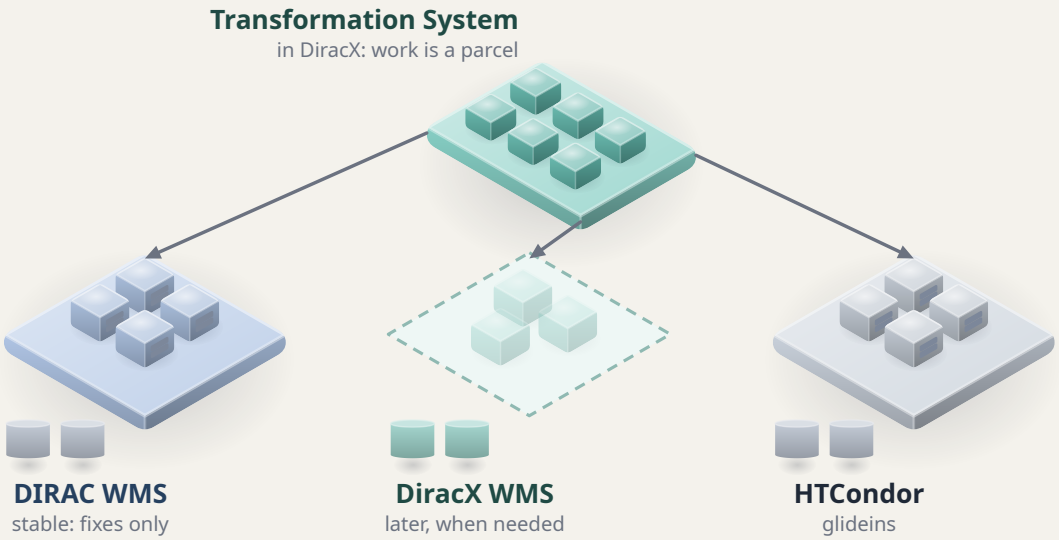
WHY IT CHANGES		WHAT SITES NOW OFFER		WHAT THE WMS MUST DO
AI expected productivity gains drive the market	→	GPUs, before they can be pledged or accounted	→	**Match on capabilities; run CPU and GPU work side by side**
Energy and hardware cost 2022 energy shock, climate targets, AI-driven prices	→	ARM next to x86: first WLCG ARM pledges in 2025	→	**One image and one match per architecture**
More cores per node clock speed stopped rising around 2005	→	Whole nodes (HPC), multicore slots (grid)	→	**Fill nodes, not single slots**
Identity X.509 gives way to the tokens industry uses	→	CEs that accept tokens, or only tokens	→	**Pilots submitted with tokens (DIRAC v8.0); DiracX built on tokens**

In DIRAC: temporary solutions, to keep it stable. **In DiracX**: the long-term solutions, designed for this.

1 · Starting from the transformation system

What it changes for workload management

The transformation system sees compute backends



each backend keeps its own data and its own data model

The transformation system (Tuesday's talks) sends parcels to compute backends. **The DIRAC WMS is one of them**, with its own data, as it is.

What separate backends give us

A stable DIRAC

The DIRAC WMS keeps its data and its behaviour. No large change: fixes and performance only, until communities move away from it.

A DiracX free of the legacy model

DiracX does not inherit DIRAC's jobs, tables or states. Work is a **parcel**; the WMS behind it is designed for it.

No rush for a DiracX WMS

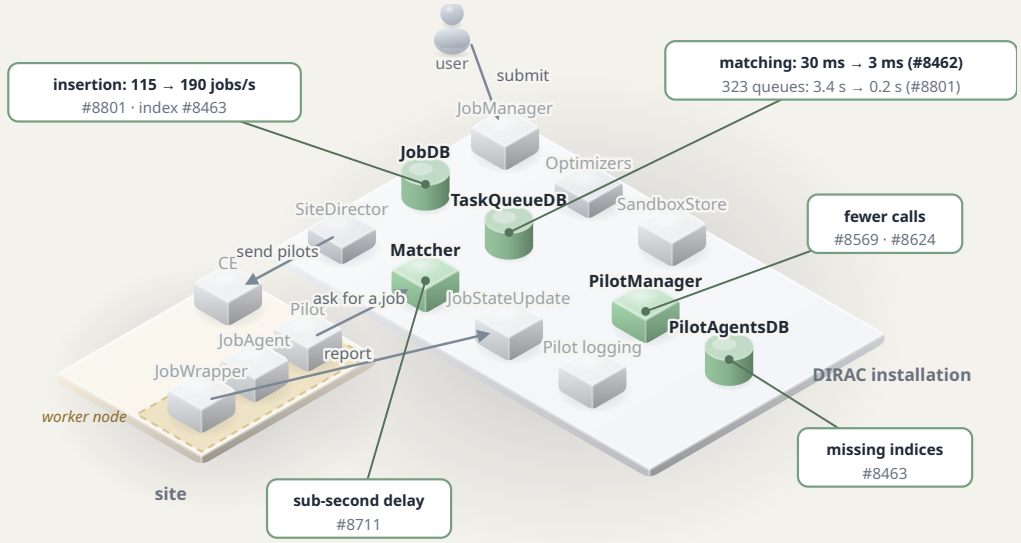
Most communities can rely on their existing WMS for now: DIRAC, or their own (see the CMS talk just before).

> The DiracX WMS is not a priority. It will move slowly, in parallel, and get a plan when the transformation system needs a native backend.

2 · DIRAC: fixes and performance

What keeps it running while we migrate

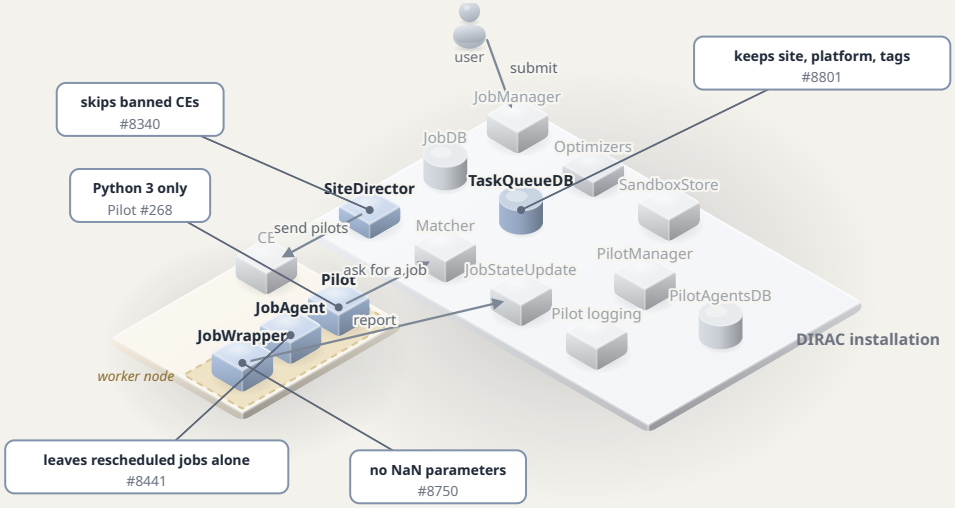
Faster



#8462 #8801 #8711 #8463 #8569 #8624 #8524

Several fixes made the DIRAC WMS faster, for installations at v9 and above. Most of these numbers need v9.1.

More reliable



Task queues keep their restrictions · #8801, v9.1.20

A race could leave a task queue briefly without its site rows: a pilot at any site could then match jobs meant for other sites.

Banned CEs skipped · #8340, v9.0.2

The SiteDirector kept trying to submit pilots to banned CEs.

Rescheduled jobs left alone · #8441, v9.0.17

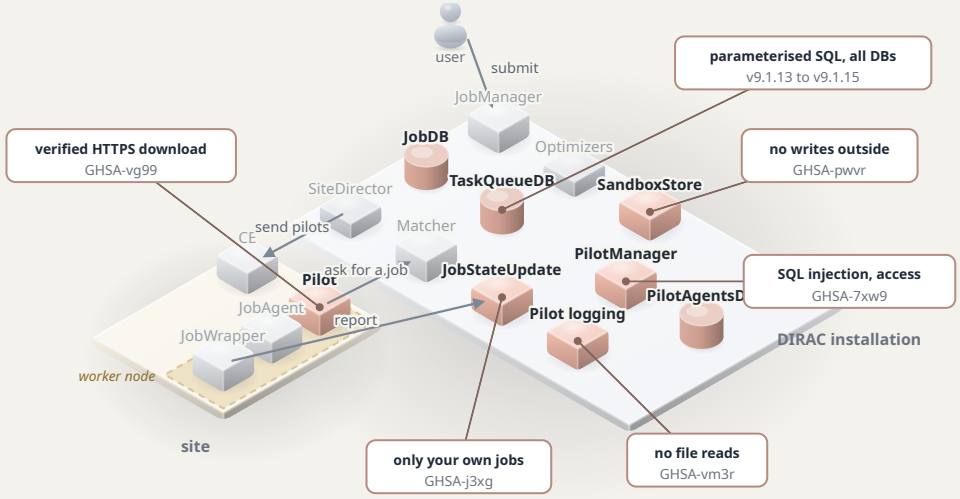
A job rescheduled after an early failure could already run elsewhere when the JobAgent, minutes later, marked it Failed.

No NaN parameters · #8750, v9.1.16

Jobs shorter than the first Watchdog cycle reported NaN: invalid JSON, so DiracX rejected the whole update.

Python 3 only pilot · Pilot #268 DUW11

Safer



Job state updates · GHSA-j3xg, high

Any authenticated user could change any job's owner, group or VO, or force its status.

Sandbox store · GHSA-pwvr, critical

A file name ending in `../` could write anywhere the service user can.

Pilot logging · GHSA-vm3r, critical

A crafted file name could read any file on the server.

Pilot manager · GHSA-7xw9, high

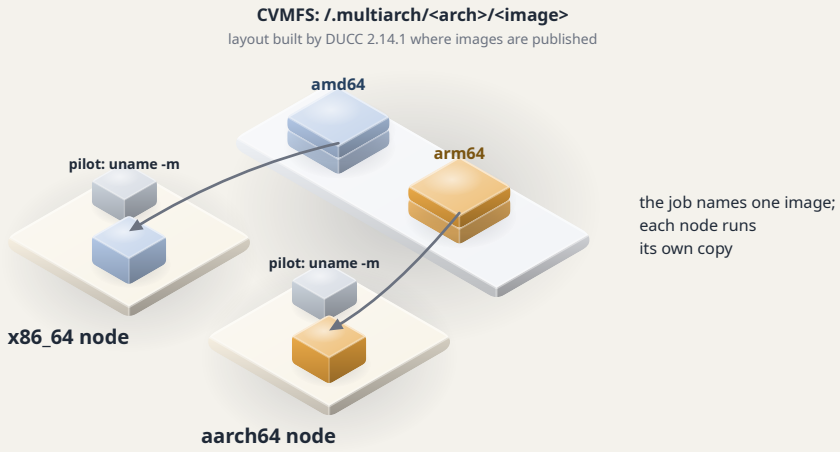
Parameters reached SQL unescaped, without access control.

Pilot download · GHSA-vg99, high

The pilot code was fetched over HTTPS without checking the certificate, then run.

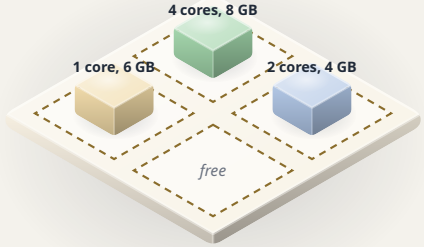
First three fixed in v9.1.20, v9.0.26, v8.0.85; last two in v9.1.10, v9.0.22. All WMS SQL parameterised in v9.1.13 to v9.1.15.

One image for every architecture



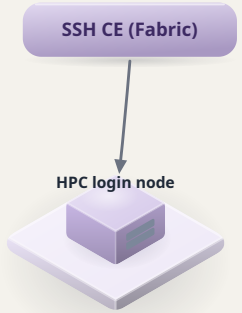
- Sites now offer x86_64 and aarch64 nodes. A job names one container image; the pilot reads the node's architecture and runs the matching copy. #8456, open, to certify `DUW11`.
- Publishing needs CVMFS 2.14 (DUCC 2.14.1); any CVMFS client reads the result.
- **Still missing in DIRAC: alternatives.** A job states one platform and one set of requirements. It cannot say "x86_64, or aarch64, or a GPU of type A". The DiracX matcher is designed for this.

Sharing a node, reaching an HPC



one multi-core slot

dashed: a cgroup2 limit per job



its batch system runs the pilots

- **CPU and memory limits** (v9.0) DUW11. Memory requests count when matching and when sharing a multi-core slot (#8366, #8414); cgroup2 keeps each job within its share (#8110).
- **HPC through SSH** (v9.0) DUW11. The SSH computing element, rewritten on Fabric, submits through an HPC login node to its batch system (#7703).

Time left in the slot: before and after



An example: an MC job sized for 183 events produced 150, was killed, and its output was never uploaded.

One source of truth since v9.1.4 (#8479): the pilot reads the time left once, jobs read it from the pilot configuration, even inside containers.

Time left in the slot: configuration

`/Systems/WorkloadManagement/JobWrapper`, or per job in the JDL:

```
StopMargin = 300              # seconds kept for the uploads
# graceful stop, off unless set; example for Gauss
StopSigRegex = ^\S*python\S*\s+\S*/gaudirun\.py\s.*/options/Gauss/
StopSigNumber = 10            # SIGUSR1
StopSigFinishWork = 16740     # CPU work to wind down
StopSigStartWork = 8370       # minimum work before stopping
```

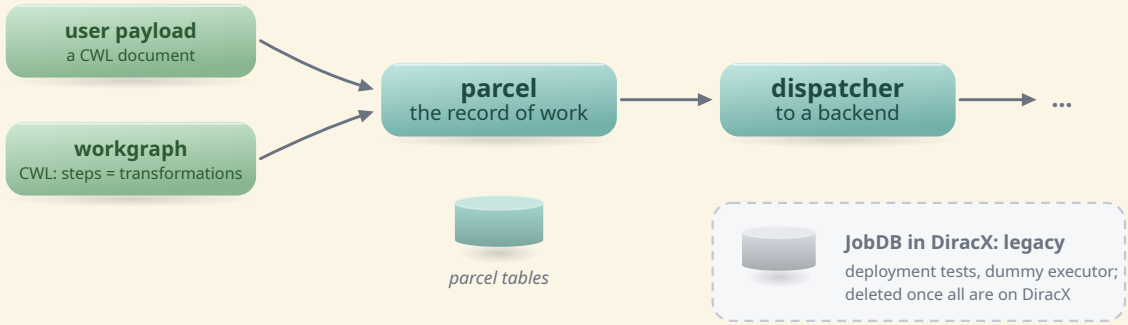
- **StopMargin** is wall clock: uploading takes the same time on any node. It is subtracted once, before matching.
- **StopSig*** thresholds are in CPU work, divided by the node's power: one value fits slow and fast nodes.
- The application must handle the signal (Gauss: `StopOnSignal`). The Watchdog still stops it when the budget is spent.

In **v9.1.21** only. Details: "Allocated time management" in the Administrator Guide. CPU work is normalised with DB12: next talk.

3 · DiracX: parcels, not jobs

And the proofs of concept around the future WMS

No jobs in DiracX



- A user payload is a parcel, like a bundle of transformation inputs. A failed parcel is final: resubmitting creates a new one.
- We do not port job monitoring or the job manager `DUW11`.

- The JobDB in DiracX is legacy. It receives job status updates from DIRAC, to **test that a DiracX deployment works**.
- It also feeds the **dummy job executor** (diracx#995): jobs go `Waiting → Matched → Running → Done` without running anything. A stand-in backend, to test the transformation system end to end.
- **Once everyone is on DiracX, it is deleted.**

CWL describes all the work

```yaml
cwlVersion: v1.2
class: CommandLineTool
$namespaces: {dirac: https://diracgrid.org/cwl#}
hints:
  dirac:Job: {}        # a user payload
baseCommand: [python, fit.py]
inputs:
  data:                # "lfn:/vo/..." on the grid,
    type: File         # a local file on a laptop
    inputBinding: {position: 1}
outputs:
  plots: {type: File, outputBinding: {glob: "*.png"}}
```

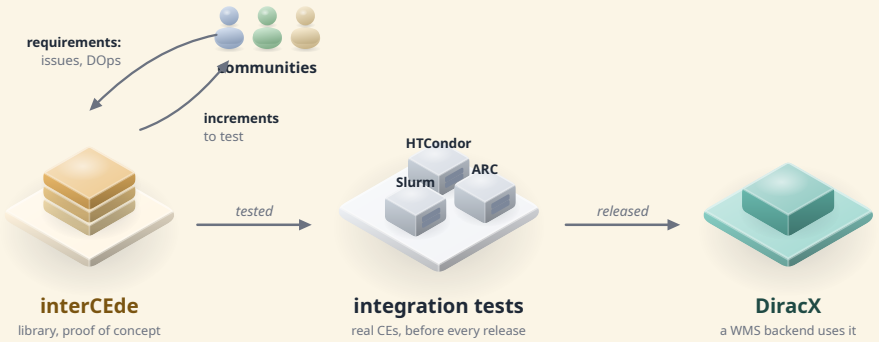
Draft syntax (DX-ADR-007). Runs as is with `cwltool`.

- **One format** for a user payload, a workgraph and each transformation: CWL v1.2.
- DiracX settings are `dirac:` **hints**: the document stays plain CWL.
- **For a WMS backend**: requirements come from the hints; the job wrapper resolves `lfn:` and `sandbox:` inputs and reports a status per input; DiracX registers the outputs.
- **Yesterday**: hands-on sessions to convert DIRAC workflows to CWL.

With the CWL community: cwltool#2302 path mapper, cwltool#2166 Apptainer sandboxes, schema_salad#1064: merged. cwl-utils#446 QuickJS engine: in review.

interCEde: computing elements, explored from use cases

- **An exploratory project**, from the use cases DIRAC covers, and those it does not but should.
- **Its own repository**: integration tests against real CEs, so that a change cannot break a production instance.
- **Ends with ADRs and a first implementation**; the ADRs are merged once that implementation works.



Open call: if you work with DIRAC computing elements, test the increments of interCEde and tell us what does not fit your needs, or what DIRAC does not cover: in an issue, or at a DOps.

A matcher for the DiracX WMS

Why

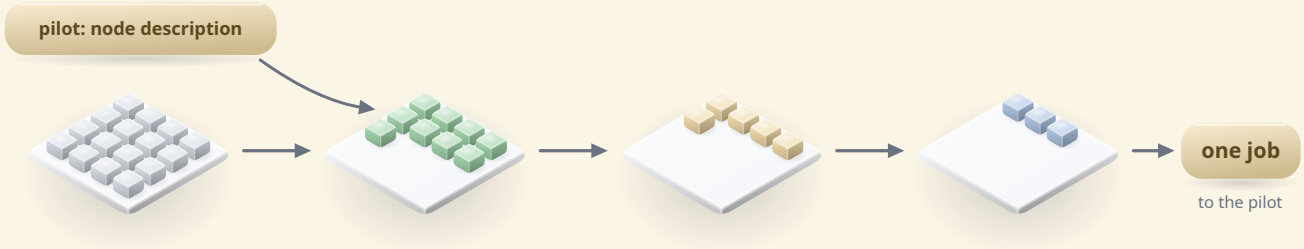
- DIRAC's matcher will not be fast enough for what is coming for some communities, such as LHCb.
- It cannot describe heterogeneous needs: a payload that runs on GPU A, *or* GPU B, *or* CPU C.

The proof of concept (poc-dirac-job-matchmaking)

- A deliberately simple Python matcher, and load tests on LHCb-shaped data to compare solutions.
- The solution must be easy to deploy and to rebuild: chaos tests with Redis switched off.

> Next: matching in Redis, measured against this baseline. Then an ADR, then the implementation in DiracX.

What the matcher POC does today

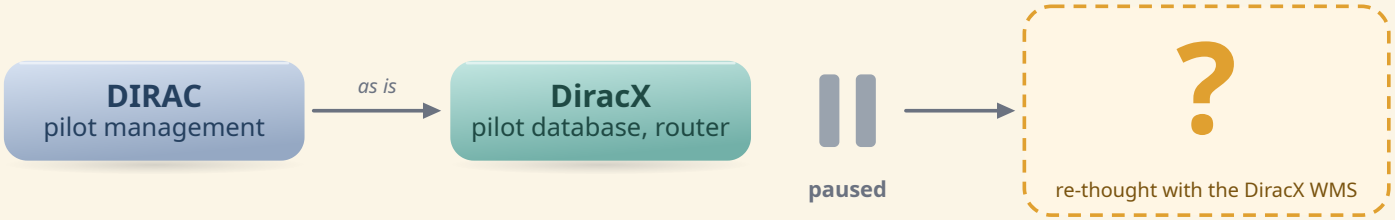


one request at a time, in Python: every waiting job is checked

```yaml
# a job's requirements (excerpt)
matching_specs:      # a list: alternatives
  - cpu:
      num-cores: {min: 1, max: 4}
      architecture: {name: x86_64}
    gpu:
      count: {min: 1, max: 1}
      vendor: nvidia
      compute-capability: {min: 8.0}
    tags: cvmfs:lhcb & (os:el9 | os:alma9)
```

```yaml
# the node a pilot describes (excerpt)
site: LCG.CERN.cern
cpu:
  num-cores: 8
  ram-mb: 32768
  architecture: {name: x86_64}
gpu: {count: 1, vendor: nvidia,
      compute-capability: 8.0}
tags: [cvmfs:lhcb, os:el9, gpu:nvidia]
```

Pilot management: port paused



We started porting pilot management as it is: a pilot database and router landed (diracx#570) DUW11.

We learned the data model could be much better; changing it would mean large changes in DIRAC, which we keep stable.

So the port is paused: registration, pilot tokens and logging wait for the DiracX WMS backend.

> Pilots belong to the DiracX WMS backend, which comes after the transformation system. The DIRAC pilot keeps being patched.

The job wrapper: one for all of us

Today there are several

- DIRAC's JobWrapper: pre-processing, payload, post-processing; an offline variant for push.
- The `dirac-cwl` wrapper, experimental in DIRAC.
- Executor prototypes in DiracX.

Each community also adds its own steps around the payload.

Requirements to gather

- Inputs: data, sandboxes, protocol access.
- Time left, graceful stop, containers, architectures.
- Nodes without outbound network.
- Reporting: status, parameters, logs.
- What happens when an upload fails.

We are looking for someone to gather the requirements and the feedback, build a proof of concept, then design the ADR, working alongside someone who has been through the process once. Come and talk to us this week.

Take-aways

THE MODEL

- The transformation system sends parcels to compute backends; the DIRAC WMS is one of them.
- DIRAC stays stable; DiracX is free of DIRAC's job model.

FOR INSTALLATIONS

- Upgrade to v9.1: faster matching and job insertion, time-left fixes (v9.1.21), security patches.
- Multi-architecture images: #8456, soon.

FOR DIRACX

- No jobs: parcels. The JobDB in DiracX only tests deployments, then goes.
- The DiracX WMS is not a priority; it will get a plan when needed.
- CWL describes everything that runs.
- interCEde and the matcher: exploratory, with tests; ADRs after a first implementation. Pilot management: paused.

WE NEED YOU

- Someone to lead the job wrapper requirements and proof of concept; your requirements for interCEde and the matcher.

Thank you

Questions and comments welcome, this week and after.

Alexandre F. Boyer · alexandre.boyer@cern.ch

github.com/DIRACGrid · diracx.diracgrid.org

